

B.Sc. (Hons) in Software Development



Ollscoil
Teicneolaíochta
an Atlantaigh

Atlantic
Technological
University

Machine Learning Based Predictive Trading and Simulation Platform

By
Michael Ferry

April 26, 2026

Minor Dissertation

**Department of Computer Science & Applied Physics,
School of Science & Computing,
Atlantic Technological University (ATU), Galway.**

Contents

1	Introduction	2
1.1	Background and Context	2
1.2	Project Overview	3
1.3	Objectives	4
1.4	Motivation	5
1.5	Document Structure	5
2	Methodology	7
2.1	Overview	7
2.2	Software Development Methodology	7
2.3	Requirements Gathering	8
2.4	Version Control and Development Tools	8
2.5	Technology Selection	8
2.6	Research Methodology	9
2.7	Testing Strategy	10
2.8	Ethical Considerations	10
3	Technology Review	12
3.1	Backend Technologies	12
3.1.1	Java 17	12
3.1.2	Spring Boot	12
3.1.3	Spring Security and JWT	13
3.1.4	PostgreSQL	13
3.1.5	Gradle	14
3.2	Frontend Technologies	14
3.2.1	React 19	14
3.2.2	Vite	14
3.2.3	Chart.js	14
3.2.4	CSS3 and Responsive Design	15
3.3	Machine Learning and Data Science	15
3.3.1	Python 3.11	15
3.3.2	Flask	16
3.3.3	Ensemble Learning with Random Forest, Gradient Boosting and XGBoost	16
3.3.4	Engineered Features and Macro Context	17
3.3.5	FinBERT for Financial Sentiment Analysis	17
3.3.6	Multi Source Sentiment Aggregation	18
3.3.7	Walk Forward Backtesting	18
3.3.8	Markowitz Portfolio Optimisation	18
3.4	External APIs	19
3.4.1	Finnhub	19

3.4.3	NewsAPI	19
3.4.4	Reddit Public JSON API	19
3.5	DevOps and Infrastructure	20
3.5.1	Docker and Docker Compose	20
3.5.2	GitHub Actions	20
3.6	Summary	20
4	System Design	22
4.1	System Architecture	22
4.2	Backend Design	24
4.2.1	Package Structure	24
4.2.2	REST API Design	24
4.2.3	Authentication Flow	25
4.2.4	Trading Engine	25
4.2.5	Prediction Logging and Evaluation	26
4.3	Machine Learning Service Design	27
4.3.1	Data Ingestion and Feature Engineering	27
4.3.2	Training the Ensemble	27
4.3.3	Sentiment Fusion	29
4.3.4	Stability Guardrail and Confidence Calibration	29
4.3.5	Market Regime Detection and Signal Generation	29
4.3.6	Backtesting and Portfolio Optimisation	30
4.4	Frontend Design	30
4.4.1	Application Structure	30
4.4.2	Dashboard Layout	30
4.4.3	State Management	31
4.4.4	Data Visualisation	32
4.5	Database Design	32
4.6	Security Design	33
4.7	Deployment Architecture	34
5	System Evaluation	35
5.1	Evaluation Against Objectives	35
5.2	Prediction Model Results	37
5.3	Strengths	38
5.4	Limitations	39
5.5	Usability	40
5.6	Ethical Considerations	40
5.7	Performance	41

6	Conclusion	42
6.1	Summary	42
6.2	Key Findings	42
6.3	Reflection	43
6.4	Future Work	44
6.5	Applied Learning Outcomes	45
6.6	Closing Remarks	45
A	Appendix	50
A.1	GitHub Repository	50
A.2	Installation Instructions	50
A.2.1	Prerequisites	50
A.2.2	Clone the Repository	51
A.2.3	Configure Environment Variables	51
A.2.4	Quick Start (All Services)	51
A.2.5	Manual Setup	51
A.2.6	Using the Application	52
A.3	API Endpoints	52
A.4	Screencast	52

List of Figures

4.1	MarketMind System Architecture.	23
4.2	Authentication and JWT Token Flow.	26
4.3	Machine Learning Prediction Pipeline.	28
4.4	MarketMind Dashboard with Live Stock Data.	31
4.5	Database Schema.	33

List of Tables

5.1	Prediction model performance over the 150 day backtest (Sep 2025 – Apr 2026). The Δ vs Baseline column reports the ensemble’s mean absolute percentage error relative to the persistence baseline; negative values indicate the ensemble is worse than the baseline on magnitude.	37
A.1	MarketMind API Endpoint Summary	52

Chapter 1

Introduction

1.1 Background and Context

Every second, financial markets produce vast quantities of data, from split second price changes and order book updates to the latest economic news and what is trending on social media. For decades, only large institutions such as investment banks, hedge funds and proprietary trading firms had the means to process and react to all this information as it happened. They could afford the expensive infrastructure and hire teams of quantitative analysts to build sophisticated trading systems. Meanwhile, ordinary investors and smaller traders were left working with delayed data, basic charting tools and no access whatsoever to proper forecasting models.

However, the landscape is changing rapidly. Advances in machine learning, cloud computing and the proliferation of open financial data APIs have begun to level the playing field. Models that detect patterns in time series data can now be trained and deployed on a standard laptop. Affordable real time and historical market data is available to developers through REST APIs from providers such as Finnhub [1] and Yahoo Finance [2]. These developments have created a genuine opportunity to build a platform that brings institutional grade financial analysis tools to a much broader audience.

The rise of retail investing has further accelerated this trend. Platforms such as Robinhood, eToro and Trading 212 have made it easier than ever for individuals to participate in the stock market. However, these platforms offer limited forecasting capabilities and almost no integration with machine learning models. There is a clear gap in the market for an accessible tool that combines live market data, predictive analytics and simulated trading in a single cohesive application.

This project, named MarketMind, was built to address that gap. It provides a full stack trading simulation platform that integrates real time stock data, ma-

chine learning price predictions, news sentiment analysis and paper trading in a responsive web application. The platform is designed for educational and research purposes, allowing users to explore market dynamics and test trading strategies without risking real capital.

1.2 Project Overview

MarketMind is a full stack predictive trading and simulation platform built using a microservices architecture. The system is composed of three principal services that communicate over HTTP. A Java Spring Boot backend serves as the central API layer, handling user authentication, trade execution, portfolio management, prediction logging and data aggregation from external sources. A Python Flask microservice is responsible for the machine learning prediction engine, which combines a three model ensemble of Random Forest, Gradient Boosting and XGBoost regressors with FinBERT powered sentiment analysis to generate forward looking return forecasts with calibrated confidence scoring. A React frontend built with Vite provides the user facing dashboard, which displays live stock prices, historical charts, prediction cards, news and social sentiment feeds, an interactive trading panel and a Markowitz portfolio optimiser.

The platform pulls live stock quotes from the Finnhub API and historical price data from Yahoo Finance. News articles relevant to a given stock are retrieved through the NewsAPI service and social discussion is gathered from six finance focused subreddits. Both text streams are scored by FinBERT, a BERT model fine tuned on financial text and fused into a single sentiment signal that feeds back into the prediction model. All of these data streams converge in the dashboard, giving the user a comprehensive view of a stock's current state, recent history, predicted direction, prevailing sentiment and an honest readout of how confident the model is in its own forecast.

Users register and log in through a secure authentication system built on Spring Security with BCrypt password hashing and JSON Web Token (JWT) session management. Once authenticated, they can search for stocks, view real time prices, examine historical price charts, receive machine learning predictions and execute simulated buy and sell orders against a virtual cash balance. Their portfolio, including open positions, total value and trade history, is tracked and displayed in real time.

1.3 Objectives

The project was set out to achieve the following objectives, each of which maps directly to a measurable outcome.

1. Develop a Java Spring Boot REST API to manage system operations, including user registration, authentication, trading and portfolio tracking.
2. Integrate external APIs for live and historical data, specifically Finnhub for real time quotes, Yahoo Finance for historical prices and NewsAPI for financial news.
3. Design and implement a PostgreSQL database to persist user accounts, credentials, trade records, holdings, portfolio snapshots and prediction logs.
4. Implement a machine learning prediction model in Python, deployed as a Flask microservice, capable of generating short term return forecasts using a three model ensemble with calibrated confidence scoring.
5. Connect the trained ML model to the Java backend through inter service HTTP communication so that predictions are available to the frontend in real time.
6. Build a demo trading module that supports simulated buy and sell operations and tracks portfolio performance over time.
7. Create a responsive React dashboard that presents live prices, historical charts, predictions, news sentiment and portfolio data in a clean, professional interface.
8. Implement a secure authentication system using BCrypt hashing and JWT tokens with role based access control.
9. Evaluate the prediction model's accuracy through automated backtesting and quantitative metrics including mean absolute error, mean absolute percentage error and directional accuracy, benchmarked against naive baselines (Always-Up, Always-Down and one-step persistence).
10. Document all design, architectural and ethical decisions comprehensively in this dissertation.

1.4 Motivation

The motivation for this project stems from a personal interest in both software development and financial markets. The stock market represents one of the most data rich environments in existence, and the challenge of building a system that can ingest, process and make sense of that data in real time is a compelling software engineering problem.

From an academic perspective, the project provided an opportunity to work across the full technology stack and integrate several distinct disciplines. Building the backend required expertise in Java, Spring Boot, RESTful API design, relational databases and security. The machine learning service demanded knowledge of Python, statistical modelling, data engineering and API design. The frontend required proficiency in React, state management, data visualisation with Chart.js and responsive CSS. Bringing all of these components together into a flowing cohesive, working system was a significant technical challenge that exercised skills developed across the entire degree programme.

The project also has practical relevance. Retail investing has grown enormously in recent years, and there is genuine demand for tools that help individuals make more informed decisions. While MarketMind operates in a simulated environment and is explicitly not intended for real trading, the architecture and techniques it demonstrates could be extended into a production system. The prediction model, built on a tree based ensemble with sentiment fusion, provides a foundation upon which more sophisticated approaches such as LSTM or transformer based models could be layered.

Finally, the project gave an opportunity to explore the ethical dimensions of financial technology. Prediction models carry inherent risks if users treat them as reliable guides to real world trading. The platform includes clear disclaimers throughout and operates exclusively with virtual funds. These considerations are discussed further in the System Evaluation chapter.

1.5 Document Structure

The remainder of this dissertation is organised as follows.

Chapter 2 describes the methodology adopted for the project, covering both the software development approach and the research methods used to inform design decisions.

Chapter 3 presents a comprehensive technology review of the frameworks, libraries, APIs and tools used in the project, grounded in the relevant academic and industry literature.

Chapter 4 details the system design, including the overall architecture, database

schema, API layer, machine learning pipeline, frontend structure and security implementation.

Chapter 5 evaluates the completed system against the objectives listed above, presents quantitative results from the prediction model and discusses the strengths, limitations and ethical considerations of the platform.

Chapter 6 concludes the dissertation by summarising the key findings, reflecting on the development process and identifying opportunities for future work.

The source code for the project is available on GitHub at <https://github.com/michaelferry25/MachineLearningBasedPredictiveTradingPlatform>. A brief screencast that demonstrates the application is also included in the repository. The appendix at the end of this document contains the GitHub link and installation instructions for compiling, deploying and running the project.

Chapter 2

Methodology

2.1 Overview

This chapter describes the approach taken to plan, develop, test and deliver the MarketMind platform. The methodology encompasses both the software development process and the research methods used to inform technical decisions throughout the project lifecycle.

2.2 Software Development Methodology

An Agile, iterative approach was adopted from the outset. The project was too large and too uncertain to follow a strict waterfall process. Requirements evolved as technical constraints became clearer, and certain features, such as the prediction model and news sentiment integration, required experimentation before a final design could be settled on.

Development was structured around two-week sprints, each with a defined set of deliverables. At the start of each sprint, a brief planning session identified the tasks to be completed and their priorities. At the end of each sprint, the working software was reviewed against the plan and any outstanding issues were carried forward into the next iteration. This cadence ensured that progress was steady and that any problems were caught early rather than snowballing into larger issues later in the project.

Weekly meetings with the project supervisor provided regular feedback and course corrections. These meetings followed a consistent structure. Progress since the last meeting was demonstrated, any blockers or design decisions were discussed and a plan for the coming week was agreed upon. Minutes were kept informally but consistently, and the supervisor had ongoing access to the GitHub repository to review code between meetings.

2.3 Requirements Gathering

The initial requirements were established during the proposal phase in the first two weeks of the project. The primary sources of requirements were the project brief, the module learning outcomes and an analysis of existing trading platforms and fintech applications.

A feature list was drawn up early and refined through discussion with the supervisor. The features were grouped into core requirements, which the project could not be considered complete without and stretch goals, which would add value if time permitted. Core requirements included live price retrieval, historical charting, a prediction model, simulated trading, user authentication and portfolio tracking. Stretch goals included news sentiment analysis, candlestick charting, a settings page and prediction accuracy metrics, all of which were ultimately delivered.

No formal user stories or storyboarding tools were used. Instead, a simple markdown checklist in the repository's README served as the task tracker. This lightweight approach suited the solo nature of the project and avoided the overhead of tools like Jira or Trello, which are better suited to team-based work.

2.4 Version Control and Development Tools

Git and GitHub were used for version control throughout the project. All code was committed to a single repository organised into three top-level directories corresponding to the three services (backend, frontend, ml-service). Commits were made frequently, typically several times per day during active development, with descriptive messages that referenced the feature or fix being worked on. The commit history provides a detailed record of the development timeline and can be used to verify that work was spread consistently across the academic year.

The development environment consisted of IntelliJ IDEA for the Java backend, Visual Studio Code for the React frontend and Python ML service and a terminal for running Docker, Gradle and npm commands. A single dev.sh shell script was created to start all three services simultaneously, streamlining the local development workflow.

2.5 Technology Selection

Technology choices were informed by a combination of prior experience, industry relevance and suitability for the task at hand.

Java and Spring Boot were chosen for the backend because they represent the dominant enterprise stack in the Irish and European job market, and because

Spring Boot offers mature support for RESTful APIs, security, database integration and scheduling out of the box. Gradle was used as the build tool for its flexibility and speed compared to Maven.

Python and Flask were selected for the machine learning microservice because Python is the de facto language for data science and machine learning work. Flask was preferred over FastAPI for its simplicity and tight integration with the scientific Python ecosystem, where the bulk of the ML service's work happens.

React with Vite was chosen for the frontend because React is the most widely used JavaScript framework for single-page applications, and Vite provides a significantly faster development experience compared to Create React App. Chart.js was selected for data visualisation due to its simplicity and the quality of its output.

PostgreSQL was chosen as the relational database because of its robustness, widespread use in production systems and excellent Spring Data JPA support. The Finnhub API was selected for live stock data after comparing it with Alpha Vantage and Twelve Data on the basis of rate limits, data quality and documentation. Yahoo Finance was used for historical data because it provides free access to a deep archive of daily price data without requiring an API key.

2.6 Research Methodology

The research component of the project focused on understanding the current state of the art in stock price prediction, evaluating the strengths and weaknesses of different modelling approaches and identifying the most appropriate techniques for the scope and timeline of the project.

Academic papers on time series forecasting, ensemble learning, gradient boosting, transformer based language models and financial sentiment analysis were reviewed through Google Scholar and the IEEE Xplore digital library. Industry resources, including API documentation, framework guides, the scikit-learn user guide and the XGBoost documentation, were also consulted extensively. All sources are cited in the Technology Review and References sections.

The prediction model was developed using an empirical, experiment driven approach. An initial prototype using a weighted moving average with trend and momentum components was built first to validate the end to end pipeline from data ingestion through Flask serving. Once the plumbing was proven, the model was rebuilt around a three way ensemble of Random Forest, Gradient Boosting and XGBoost regressors trained to predict five day forward returns rather than raw next day prices. Predicting returns was chosen over predicting prices because returns are stationary, which is a basic requirement for tree based regressors that struggle when the target drifts over time. Sentiment analysis was upgraded from TextBlob to FinBERT after early testing showed that TextBlob misclassified fi-

nancial language with domain specific connotations such as "cutting", "dilution" and "headwinds". A walk forward backtester was built to evaluate strategy performance under realistic conditions, and a Markowitz portfolio optimiser was added to extend the platform beyond single stock prediction.

All performance metrics were reported alongside naive baselines, specifically Always-Up, Always-Down and a one-step persistence forecast, following the naive benchmark methodology recommended by Hyndman and Athanasopoulos [3]. Reporting the ensemble's hit rate and MAPE against these baselines rather than in isolation was a deliberate choice so that headline figures are always read in context, and so that any apparent edge is tested against the simplest possible alternative rather than taken at face value.

2.7 Testing Strategy

Testing was approached pragmatically. The backend was covered by unit tests written with JUnit 5, focusing on the service layer where business logic resides. Integration tests verified that the REST controllers returned correct responses for valid and invalid inputs. The authentication flow, including registration, login and JWT validation, was tested end-to-end using both automated tests and manual Postman requests.

The Python ML service was tested using Pytest, with test cases covering the prediction function for stocks with sufficient data, stocks with insufficient data and invalid symbols. The health endpoint was also tested to ensure the service reported its status correctly.

Frontend testing was carried out manually due to the visual and interactive nature of the React components. Each page and feature was tested in multiple browsers (Chrome, Firefox, Safari) and at multiple screen widths to verify responsiveness. The authentication flow, trading panel, portfolio updates and chart rendering were all tested manually against the running backend and ML services.

A GitHub Actions CI pipeline was configured to run backend and ML service tests automatically on every push to the main branch. This ensured that regressions were caught immediately and that the codebase remained in a deployable state throughout the project.

2.8 Ethical Considerations

The project raises several ethical considerations that were addressed during design and development.

First, MarketMind operates exclusively as a simulation platform using virtual

funds. This is made clear to users through disclaimers on the landing page, the footer and throughout the user interface. The platform is not intended for and must not be used for real financial trading decisions.

Second, all data collected by the platform is limited to email addresses and display names provided voluntarily during registration. No personal financial data is collected or stored. Passwords are hashed using BCrypt and are never stored in plaintext. JWT tokens expire after a defined period to limit the window of exposure in the event of a token leak.

Third, all market data used by the platform is publicly available through the Finnhub and Yahoo Finance APIs under their respective terms of service. No proprietary or restricted data is used.

Fourth, the prediction model is presented honestly. The model architecture, the FinBERT sentiment fusion and the calibrated confidence range are all surfaced through the API and shown in the user interface alongside every prediction. Confidence scores are calibrated against validation error and historical hit rate rather than being purely cosmetic, and they are intentionally bounded between 35 and 94 percent so that the model never claims certainty it cannot defend. This transparency is important given the potential for harm if users were to treat the model's output as investment advice.

Chapter 3

Technology Review

This chapter examines the technologies, frameworks, APIs and libraries used in the MarketMind platform. Each technology is discussed at a conceptual level, with reference to the academic and industry literature that informed its selection. The review is organised by the layer of the system to which each technology belongs.

3.1 Backend Technologies

3.1.1 Java 17

Java remains one of the most widely used programming languages in enterprise software development. The 2024 JetBrains Developer Ecosystem Survey found that Java was the primary language for backend development in over 30 percent of organisations surveyed [4]. Java 17, a Long-Term Support (LTS) release, was chosen for this project because it provides modern language features such as sealed classes, pattern matching for instanceof and text blocks, while retaining the stability and backward compatibility that the Spring ecosystem depends upon [5].

Java’s strong typing, mature tooling and extensive standard library make it well suited to building reliable, maintainable backend services. The Java Virtual Machine (JVM) also provides excellent runtime performance and garbage collection, which are important considerations for a service that may handle concurrent API requests from multiple users.

3.1.2 Spring Boot

Spring Boot is an opinionated framework for building production-ready Java applications with minimal configuration [6]. It builds on the Spring Framework and provides auto-configuration, embedded web servers and a convention-over-configuration approach that dramatically reduces boilerplate code.

In this project, Spring Boot serves as the foundation for the entire backend service. Spring Web provides the RESTful controller layer. Spring Data JPA handles database access through repository interfaces. Spring Security manages authentication and authorisation. Spring Scheduling powers the automated prediction logging that runs at configurable intervals.

The choice of Spring Boot was also informed by its dominance in the Irish software industry. Familiarity with this framework is a genuine advantage in the job market, and the project provided an opportunity to develop a deep, practical understanding of its capabilities.

3.1.3 Spring Security and JWT

Spring Security is the standard framework for securing Spring-based applications [7]. It provides a comprehensive and highly configurable authentication and authorisation model. In MarketMind, Spring Security is configured to use a stateless session model based on JSON Web Tokens (JWT).

When a user registers, their password is hashed using the BCrypt algorithm, which incorporates a salt to defend against rainbow table attacks [8]. Upon login, the server verifies the password against the stored hash and, if successful, generates a JWT containing the user's email and an expiry timestamp. This token is returned to the client and included in the Authorization header of subsequent requests. A custom `JwtAuthenticationFilter` intercepts each request, validates the token and populates the Spring Security context with the authenticated user's details.

This approach was chosen over traditional session-based authentication because it aligns with the stateless, microservices-oriented architecture of the platform. Tokens can be validated without querying a session store, which simplifies deployment and reduces coupling between services.

3.1.4 PostgreSQL

PostgreSQL is an open-source, object-relational database system known for its reliability, data integrity and standards compliance [9]. It consistently ranks among the most popular databases worldwide and is the default choice for many Spring Boot applications.

In MarketMind, PostgreSQL stores user accounts (with hashed passwords), prediction logs (with timestamps, predicted prices, actual prices and error metrics) and is the persistence layer for the Spring Data JPA repositories. The database schema was kept deliberately simple to match the scope of the project, but PostgreSQL's support for advanced features such as JSON columns, full-text search and window functions means that the schema could be extended significantly without migrating to a different database.

3.1.5 Gradle

Gradle is a build automation tool that uses a Groovy or Kotlin DSL for build scripts [10]. It was chosen over Maven for its faster build times, more concise configuration and better support for incremental compilation. The backend's `build.gradle` file defines all dependencies, including Spring Boot starters, the PostgreSQL JDBC driver and testing libraries.

3.2 Frontend Technologies

3.2.1 React 19

React is a JavaScript library for building user interfaces, developed and maintained by Meta [11]. It introduced the concept of a virtual DOM, which allows efficient updates to the browser's DOM by computing the minimal set of changes needed when state changes. React's component-based architecture encourages code reuse and separation of concerns.

MarketMind's frontend is built entirely in React, using functional components and hooks (`useState`, `useEffect`, `useMemo`) for state management and side effects. The application is structured as a single-page application (SPA) with hash-based routing. Each page or feature is encapsulated in its own component, from `PopularStocks` and `PredictionCard` to `TradingPanel` and `PortfolioOptimizer`.

React 19 was selected as the most recent major release at the time of development. It builds on React 18's automatic batching and concurrent rendering improvements, both of which keep the dashboard responsive when multiple state updates occur in rapid succession, as happens when live price data, predictions and sentiment are fetched simultaneously.

3.2.2 Vite

Vite is a next-generation frontend build tool created by Evan You, the author of Vue.js [12]. It uses native ES modules during development to provide near-instantaneous hot module replacement (HMR), and Rollup for optimised production builds. Vite was chosen over Create React App because of its significantly faster startup and rebuild times, which made the development experience noticeably smoother.

3.2.3 Chart.js

Chart.js is an open-source JavaScript charting library that renders charts on an HTML5 canvas element [13]. It supports a wide range of chart types, including

line, bar, doughnut and scatter charts and provides built-in support for animations, tooltips and responsive sizing.

In MarketMind, Chart.js is used extensively. The HistoricalChart component renders a line chart of closing prices over a configurable time period. The PerformanceChart component visualises portfolio value over time. The AnalyticsHub page uses line, bar and doughnut charts to display liquidity scores, sector fund flows and factor rotation data. The CandleChart component renders candlestick data for the LiveMarket page.

Chart.js was preferred over D3.js because of its simpler API and the fact that the charting requirements of this project did not demand the low-level control that D3 provides. For the types of charts needed, Chart.js delivered clean, professional output with minimal configuration.

3.2.4 CSS3 and Responsive Design

The frontend is styled using custom CSS3 with no CSS framework or preprocessor. This decision was made deliberately to demonstrate a thorough understanding of modern CSS, including Flexbox, CSS Grid, custom properties (variables), media queries, backdrop filters and gradient backgrounds. The dark theme that defines MarketMind's visual identity is implemented through a carefully chosen palette of deep backgrounds, accent blues and green and red indicators for positive and negative changes.

A light theme is also available, toggled through the Settings page and implemented using CSS class overrides on the body element. Responsive breakpoints ensure that the layout adapts gracefully from desktop to tablet to mobile viewports.

3.3 Machine Learning and Data Science

3.3.1 Python 3.11

Python is the dominant language in data science and machine learning, with a rich ecosystem of libraries for numerical computation, data manipulation and model development [14]. Python 3.11 was used for the ML service because of its performance improvements over earlier versions and its compatibility with all required libraries.

3.3.2 Flask

Flask is a lightweight WSGI web framework for Python [15]. It follows a minimalist philosophy, providing the essentials for building web applications (routing, request handling, response formatting) without imposing any particular project structure or ORM. Flask was chosen for the ML service because the service's HTTP layer is thin compared to its underlying numerical and modelling work, and a full featured framework like Django would have added overhead without meaningful benefit. Flask also integrates cleanly with the scientific Python ecosystem, which is where the bulk of the ML service's code lives.

Flask-CORS is used to handle Cross-Origin Resource Sharing, allowing the React frontend and the Java backend to communicate with the Flask service without browser-level restrictions.

3.3.3 Ensemble Learning with Random Forest, Gradient Boosting and XGBoost

The prediction engine is built around an ensemble of three tree based regressors trained on a common feature set, with their outputs combined using inverse mean absolute error weighting derived from a held out validation period. Ensemble methods are a well established technique for improving the stability and accuracy of supervised learning models because they reduce variance by averaging across estimators that make different kinds of errors [16].

Random Forest [17] is a bagging based ensemble of decision trees. Each tree is trained on a bootstrap sample of the training data and considers only a random subset of features at each split, which decorrelates the trees and reduces overfitting. The MarketMind implementation uses 300 trees with a maximum depth of 8 and a minimum of four samples per leaf, configured through the scikit-learn library [18].

Gradient Boosting [19] takes a different approach. Rather than training trees in parallel, it builds them sequentially, with each new tree fit to the residuals of the previous ensemble. This allows the model to focus on the examples that the existing ensemble gets wrong, but it makes overfitting easier than with Random Forest. The MarketMind configuration uses 250 estimators, a maximum depth of 4, a learning rate of 0.02 and 80 percent row subsampling to keep each tree's view of the data narrow.

XGBoost [20] is a regularised gradient boosting implementation that has become a standard tool in applied machine learning competitions and production systems. It improves on classical gradient boosting through second order gradient information, parallelisation, column subsampling and L1/L2 regularisation. The MarketMind XGBoost regressor uses 300 trees, a maximum depth of 5, a learning rate of 0.02, 85 percent row and column subsampling and L1/L2 regularisation

coefficients of 0.3 and 2.0 respectively.

The three models are trained on the same scaled feature matrix using a chronological 70 percent train, 15 percent validation, 15 percent test split. The validation period is used to compute each model's mean absolute error, and the ensemble weights are set in inverse proportion to those errors so that better performing models contribute more. This adaptive weighting matters because the relative strength of each model varies across stocks and time periods.

The ensemble predicts the percentage return five trading days ahead rather than the raw next day price. This decision was driven by two considerations. First, returns are approximately stationary while prices drift and tree based regressors handle stationary targets far better than non stationary ones. Second, a five day horizon dampens daily noise enough for the model to capture genuine trend without committing to predictions so far out that confidence collapses.

3.3.4 Engineered Features and Macro Context

The model is fed 29 engineered features spanning four categories. Twenty three are technical indicators computed from the OHLCV price history, including simple and exponential moving averages, MACD [21], Relative Strength Index [22], Bollinger Bands, Average True Range, Average Directional Index, On Balance Volume, the Stochastic Oscillator and a set of return and momentum measures. Two features capture the FinBERT sentiment score and confidence described in the next subsection. Four are macro context features derived from the CBOE Volatility Index (VIX) and the S&P 500 index, which provide a coarse readout of broader market conditions.

All features are normalised using scikit-learn's StandardScaler [18], fit on the training set and applied without leakage to the validation and test sets. Features are computed inside a `calculate_technical_indicators` method on the predictor class to keep feature engineering co located with the model code.

3.3.5 FinBERT for Financial Sentiment Analysis

Sentiment analysis in MarketMind is powered by FinBERT [23], a BERT [24] model that has been fine tuned on a corpus of financial news. FinBERT outputs separate probabilities for positive, negative and neutral sentiment, from which a polarity score in the range $[-1, +1]$ is computed as $P(\text{positive}) - P(\text{negative})$. This is meaningfully better suited to financial text than general purpose sentiment libraries such as TextBlob [25], which can misclassify domain specific phrases like "cutting costs", "beating expectations" or "diluted shares" because their everyday connotations diverge from their financial ones.

FinBERT is loaded through the HuggingFace Transformers library [26] and called in batches of sixteen articles or posts at a time, which keeps inference latency manageable. TextBlob is retained as a fallback path for environments where the FinBERT weights cannot be loaded, but the production code path uses FinBERT exclusively.

3.3.6 Multi Source Sentiment Aggregation

The sentiment service draws from two independent sources in parallel. News headlines and descriptions are fetched from NewsAPI for the previous seven days, with up to fifty articles per symbol. Social discussion is gathered from six finance focused subreddits (wallstreetbets, stocks, investing, StockMarket, options and Daytrading) using Reddit's public JSON API. Posts are weighted by engagement (upvotes plus twice the comment count) so that high signal threads carry more influence than throwaway posts. Both sources are scored by FinBERT in batch and combined with weights bounded between 30 and 70 percent based on the relative number of items returned. When the two sources agree on direction, combined confidence is boosted by ten points; when they disagree, it is penalised by ten points.

3.3.7 Walk Forward Backtesting

The backtesting engine evaluates strategy performance under realistic conditions using an expanding window walk forward methodology [27]. The model is retrained every 30 trading days on all historical data available up to that point, mirroring how a production system would behave. A long only strategy with a buy threshold of +0.3 percent predicted return and a sell threshold of -0.2 percent is simulated on the held out test window, and results are reported as Sharpe ratio, alpha relative to buy and hold, maximum drawdown, win rate and profit factor.

3.3.8 Markowitz Portfolio Optimisation

For users wanting to think beyond a single stock, the platform provides a Markowitz mean variance portfolio optimiser [28]. Given a basket of symbols, the optimiser fetches one year of daily returns, estimates the mean return vector and covariance matrix and uses scipy's SLSQP solver [29] to compute the maximum Sharpe ratio portfolio, the minimum variance portfolio, a balanced equal risk contribution portfolio and the efficient frontier. All optimisations are long only with weights summing to one. The Sharpe ratio [30] is computed using a 5 percent annual risk free rate.

3.4 External APIs

3.4.1 Finnhub

Finnhub is a financial data platform that provides real-time stock quotes, company information and market news through a RESTful API [1]. The free tier allows up to 60 API calls per minute and provides access to US stock exchanges. In MarketMind, the Finnhub API is used to retrieve the current quote for a given stock symbol, including the current price, daily high, daily low, opening price and previous close.

The Java backend communicates with Finnhub through a dedicated Finnhub-Client service class that constructs HTTP requests, parses JSON responses and handles error cases such as rate limiting and invalid symbols.

3.4.2 Yahoo Finance

Yahoo Finance provides historical stock price data through an unofficial but widely used HTTP endpoint [2]. The ML service uses this endpoint to retrieve five years of daily closing prices for a given symbol, which are then fed into the prediction model. The HistoricalPriceService in the Java backend also uses Yahoo Finance data (via the yfinance Python library) to populate the historical chart in the frontend.

Yahoo Finance was chosen because it provides a deep archive of historical data without requiring registration or an API key, making it ideal for academic and research use.

3.4.3 NewsAPI

NewsAPI is a JSON API that delivers news articles from over 150,000 sources worldwide [31]. The free developer tier provides up to 100 requests per day and returns article metadata including headlines, descriptions, source names and publication timestamps.

In MarketMind, the Java backend's NewsClient service queries the NewsAPI for articles related to a given stock's company name, while the ML service uses the same API directly to gather text for FinBERT scoring. Articles are surfaced in the React NewsSentiment component alongside their per article polarity scores.

3.4.4 Reddit Public JSON API

Reddit exposes a lightweight, unauthenticated JSON endpoint at `/r/<subreddit>/search.json` which returns recent submissions matching a query. The MarketMind sentiment

service queries six finance focused subreddits in parallel through a thread pool, retrieves the past week's submissions for the requested ticker and feeds the combined post text through FinBERT alongside the news headlines. Posts are weighted by an engagement score derived from upvotes and comment counts so that high traction discussion influences the aggregate sentiment more than low engagement noise. Reddit was chosen as a complementary signal to professional news because retail trader sentiment frequently leads or contradicts mainstream coverage, particularly for the higher beta names the platform tracks.

3.5 DevOps and Infrastructure

3.5.1 Docker and Docker Compose

Docker is a platform for building, shipping and running applications inside lightweight, portable containers [32]. Each container packages an application with its dependencies, ensuring consistent behaviour across development, testing and production environments.

Dockerfiles were created for both the Java backend and the Python ML service. A Docker Compose configuration file orchestrates all three services (backend, ML service and a PostgreSQL database), allowing the entire platform to be started with a single command. This containerised deployment model simplifies onboarding for new developers and ensures that the application runs identically on any machine with Docker installed.

3.5.2 GitHub Actions

GitHub Actions is a continuous integration and continuous deployment (CI/CD) platform integrated directly into GitHub [33]. A workflow file (`.github/workflows/ci.yml`) was configured to run the backend's Gradle test suite and the ML service's Pytest tests on every push to the main branch. This automated testing pipeline catches regressions early and provides confidence that the codebase is always in a working state.

3.6 Summary

The technologies selected for MarketMind were chosen to balance industry relevance, fitness for purpose and the opportunity to develop and demonstrate a broad range of technical skills. The combination of Java Spring Boot, Python Flask, React, PostgreSQL and Docker provides the structural backbone, while scikit-learn,

XGBoost, HuggingFace Transformers and SciPy provide the numerical and machine learning depth. Each technology is widely adopted in industry and well documented in the literature, which supports the maintainability and extensibility of the system.

Chapter 4

System Design

This chapter provides a detailed explanation of the overall system architecture, the design of each component and the interactions between them. The design was informed directly by the research conducted in the Technology Review and reflects the standards based, modular approach described in the Methodology.

4.1 System Architecture

MarketMind follows a microservices architecture with three independently deployable services communicating over HTTP. The React frontend runs on port 5173 during development and communicates with the Java Spring Boot backend on port 8080. The backend in turn communicates with the Python Flask ML service on port 5001 when predictions are requested. External data is sourced from the Finnhub API for live quotes, Yahoo Finance for historical prices and NewsAPI for financial news. The entire platform is hosted on Microsoft Azure and accessible at marketmind.cfd.

Figure 4.1 illustrates the full system architecture. This separation of concerns offers several advantages. Each service can be developed, tested and deployed independently. The Java backend handles business logic, security and data aggregation without needing to know the internals of the prediction model. The ML service can be updated or replaced with a more sophisticated model without touching the backend code. The frontend is a pure client side application that could be served from a CDN in a production deployment.

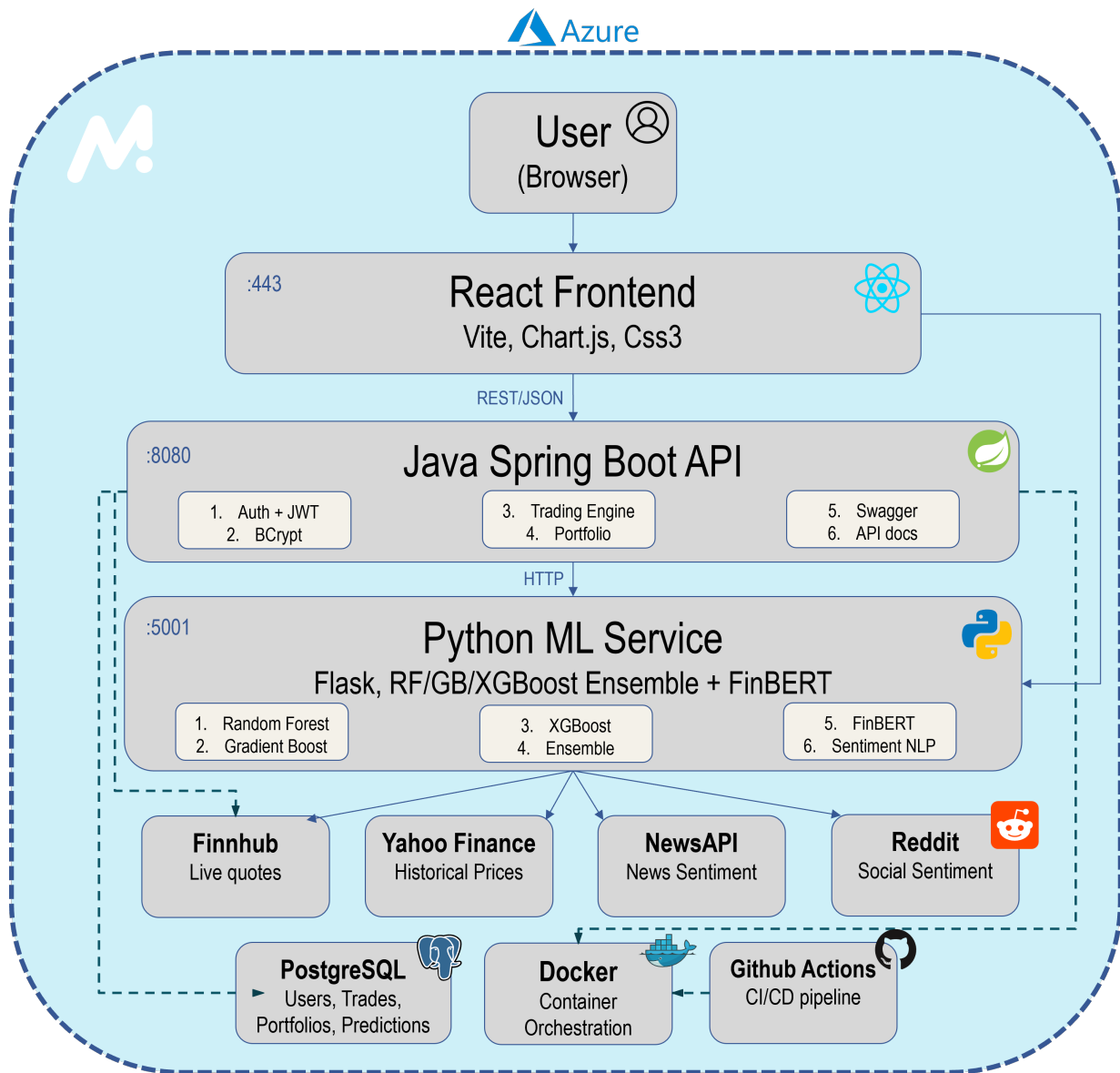


Figure 4.1: MarketMind System Architecture.

4.2 Backend Design

4.2.1 Package Structure

The Java backend is organised into a clean package hierarchy rooted at `ie.michaelferry.tradingapi`. The principal packages are as follows.

The **auth** package contains the authentication and user management layer, including `AuthController`, `AuthService`, `UserAccount` (the JPA entity), `UserRepository`, `UserRole` and a security sub-package with `JwtService`, `JwtAuthenticationFilter`, `CustomUserDetailsService`, `RateLimitInterceptor`, `RestTemplateConfig`, `WebConfig` and `SecurityConfig`.

The **auth.dto** package contains the data transfer objects for authentication requests and responses (`RegisterRequest`, `LoginRequest`, `AuthResponse`, `UserResponse`, `UpdateProfileRequest`, `ChangePasswordRequest`). These are implemented as Java records, which provide immutable data carriers with minimal boilerplate.

The **controllers** package contains the REST controllers for stock data (`StockController`), trading operations (`TradingController`) and machine learning predictions (`MLController`).

The **services** package contains the business logic classes (`StockService`, `TradingService`, `HistoricalPriceService`, `NewsService`, `SentimentService`, `CandleService`, `FinnhubClient` and `NewsClient`).

The **models** package contains the data models for API responses (`StockResponse`, `HistoricalPriceResponse`, `Portfolio` and `Trade`).

The **trading** package contains the persistence layer for the simulated trading engine (`TradeEntity`, `TradeRepository`, `HoldingEntity`, `HoldingRepository`, `PortfolioSnapshotEntity`, `PortfolioSnapshotRepository`, `UserCashBalance` and `UserCashBalanceRepository`).

The **ml** package contains the prediction logging infrastructure (`PredictionLog`, `PredictionLogRepository`, `PredictionLogService` and `PredictionScheduler`).

The **admin** package contains an administrative API protected by the ADMIN role (`AdminController` and `AdminService`) for platform statistics, user management and trade auditing.

The **learning** package contains the educational module's persistence layer (`LearningController`, `LearningProgress`, `LearningProgressRepository`, `UserXP` and `UserXPRepository`) which tracks lesson completion and experience points.

4.2.2 REST API Design

The backend exposes a RESTful API with endpoints grouped by domain. Stock data endpoints are prefixed with `/api/price`, `/api/historical`, `/api/news`

and `/api/candles`. Trading endpoints are prefixed with `/api/trading`. Authentication endpoints live under `/api/auth`. Machine learning endpoints are under `/api/ml`. Full API documentation is available through the integrated Swagger UI.

All endpoints return JSON. Cross origin requests are permitted from all origins during development using the `@CrossOrigin` annotation. In a production deployment, this would be restricted to the frontend domain.

4.2.3 Authentication Flow

The authentication system works as follows. A user submits a registration request containing their email, display name and password. The `AuthService` normalises the email to lowercase, checks for duplicates, hashes the password with `BCrypt`, creates a `UserAccount` entity and persists it to PostgreSQL. A JWT is generated containing the user's email and an expiry timestamp, and is returned along with the user's profile in an `AuthResponse` object.

For login, the user submits their email and password. Spring Security's `AuthenticationManager` verifies the credentials against the stored hash. If authentication succeeds, a new JWT is generated and returned. If it fails, a 401 Unauthorized response is sent.

Subsequent requests include the JWT in the Authorization header as a Bearer token. The `JwtAuthenticationFilter` extracts the token, validates it against the signing key, checks the expiry and populates the `SecurityContext` with the authenticated user. Protected endpoints can then access the current user through the `@AuthenticationPrincipal` annotation. Figure 4.2 illustrates this flow.

4.2.4 Trading Engine

The `TradingService` manages a per user portfolio that tracks cash balance, open holdings and trade history, all of which are persisted to PostgreSQL. When a buy order is submitted, the service verifies that the user has sufficient cash through the `UserCashBalanceRepository`, fetches the current price from the `StockService`, deducts the cost from the cash balance and either creates a new `HoldingEntity` or updates the average cost basis on an existing one. Sell orders reverse this process, checking that the user holds enough shares before executing and computing realised profit or loss against the average cost.

Every executed trade is written to the trades table as a `TradeEntity`, capturing user, symbol, type, price, quantity, total value, profit or loss and timestamp. A scheduled task periodically writes a snapshot of the user's total portfolio value to the `PortfolioSnapshotEntity` table, which provides the time series used to render the equity curve in the frontend.

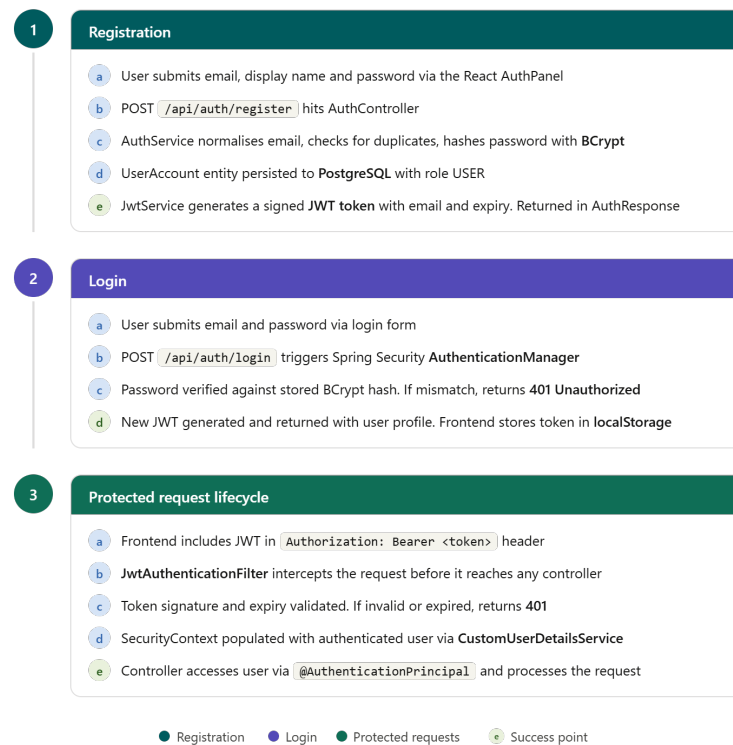


Figure 4.2: Authentication and JWT Token Flow.

The portfolio summary returned to the user is computed dynamically by fetching current prices for all held positions and combining them with the cached cash balance to give total holdings value, total portfolio value (cash plus holdings) and unrealised profit or loss for each open position.

4.2.5 Prediction Logging and Evaluation

The PredictionLog entity records every prediction made by the ML service, including the symbol, the model name, the predicted price, the current price at the time of prediction, the confidence score, the predicted direction and the prediction horizon in hours.

The PredictionScheduler runs on a configurable interval (defaulting to one hour) and requests predictions for a list of tracked symbols from the ML service. Each response is persisted as a PredictionLog entry. A second scheduled task evaluates predictions whose horizon has elapsed by fetching the actual price and computing the absolute error and percentage error.

The PredictionLogService also exposes aggregate metrics, including mean ab-

solute error, mean percentage error and directional accuracy (the proportion of predictions that correctly identified the price direction).

4.3 Machine Learning Service Design

The ML service is a Python Flask application that exposes eleven endpoints covering health checking, single and detailed predictions, batch prediction, raw sentiment retrieval, live intraday confidence updates, portfolio optimisation, walk forward backtesting, a market fear and greed index, cache management and scheduler status. The service runs on port 5001 and is called by both the Java backend (for scheduled prediction logging) and the React frontend (for user initiated requests).

Figure 4.3 illustrates the end to end prediction pipeline. The core class is `EnhancedStockPredictor` in `ml_model.py`, which encapsulates data ingestion, feature engineering, training, prediction, sentiment fusion, guardrails and confidence calibration. Predictions are cached in memory for thirty minutes to amortise the cost of training across short bursts of requests, and a background scheduler thread refreshes predictions for the eight tracked symbols (AAPL, MSFT, TSLA, AMZN, NVDA, META, NFLX, GOOGL) every hour.

4.3.1 Data Ingestion and Feature Engineering

For each prediction the service fetches five years of daily OHLCV data for the requested symbol from Yahoo Finance via the `yfinance` library, plus parallel histories for the VIX and S&P 500 indices to serve as macro context. Twenty nine features are computed per row, comprising twenty three technical indicators (moving averages, MACD, RSI, Bollinger Bands, ATR, ADX, Stochastic Oscillator, OBV, returns, momentum and 52 week position), two sentiment features sourced from FinBERT and four macro features derived from VIX and SPY. Features are normalised using `scikit-learn`'s `StandardScaler`, fit only on the training portion of the data to prevent leakage.

4.3.2 Training the Ensemble

The dataset is split chronologically into 70 percent training, 15 percent validation and 15 percent test partitions. Three models are trained on the training set: a Random Forest with 300 trees, a Gradient Boosting Regressor with 250 trees and a learning rate of 0.02 and an XGBoost regressor with 300 trees and L1/L2 regularisation. The target variable is the percentage return five trading days ahead, clipped to ± 25 percent to bound the influence of extreme moves.

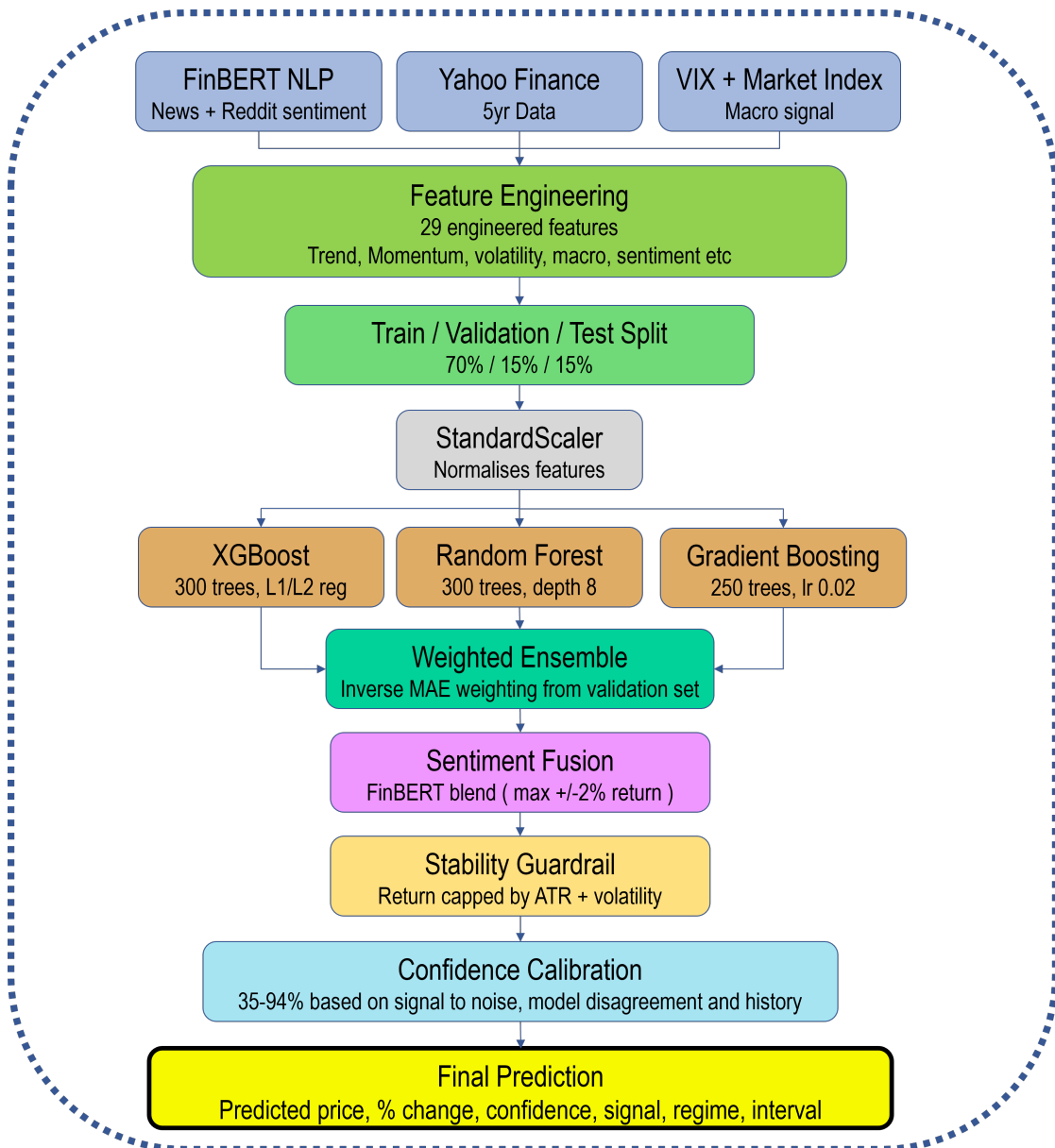


Figure 4.3: Machine Learning Prediction Pipeline.

After training, each model is scored on the validation set and the ensemble weights are set in inverse proportion to validation MAE. The weighted ensemble is then evaluated on the held out test set against a naive no move baseline (predicting zero return) to compute MAPE outperformance and directional accuracy. These metrics feed directly into confidence calibration.

A continuous learning method (`continuous_learning_update`) re-fetches the latest 5 year window and re-runs training with GridSearchCV using TimeSeriesSplit cross validation on three folds, automatically retuning the Random Forest and XGBoost hyperparameters to adapt to the most recent regime.

4.3.3 Sentiment Fusion

Sentiment enters the pipeline at two layers. First, the FinBERT polarity score and confidence are added to the feature matrix as features 24 and 25, allowing the trees to learn how sentiment interacts with price action. Second, after the ensemble produces a return prediction, a separate fusion step blends in a sentiment derived return component (capped at ± 2 percent) with a blend weight bounded between zero and forty percent. When the model and sentiment agree on direction, the blend weight is boosted by 15 percent; when they disagree, it is reduced by 45 percent. This two layer design lets sentiment shape both what the model learns and how the final forecast is adjusted at inference time.

4.3.4 Stability Guardrail and Confidence Calibration

A stability guardrail caps the magnitude of the predicted return to keep forecasts inside a realistic envelope for the stock in question. The cap is computed as the minimum of two values: 1.35 times the 95th percentile of the past 180 days of absolute daily returns, and a risk based cap derived from current ATR and volatility. The cap is bounded between 0.8 and 7 percent, and is tightened further if the ensemble has been losing to the naive baseline. Once the cap is set, the proposed return is passed through a tanh function scaled to the cap, which preserves direction and small movements but smoothly compresses extreme predictions.

Confidence is then calibrated against four signals. Signal to noise ratio (the predicted return divided by 1.5 times the validation MAE) contributes up to 22 points; model agreement (low disagreement between the three ensemble members) contributes up to 14 points; outperformance versus baseline contributes up to 10 points; sentiment alignment contributes up to 6 points. Confidence is hard clamped between 35 and 94 percent, and is forced down further if the ensemble has materially underperformed the baseline. A second pass adjusts confidence based on the symbol's historical hit rate, fetched from the backend's prediction log.

4.3.5 Market Regime Detection and Signal Generation

Each prediction is tagged with a market regime, classified into `high_volatility`, `bullish_trend`, `bearish_trend` or `range_bound` based on current ADX,

volatility and the price's position relative to the 200 day SMA. A signal (STRONG BUY, BUY, HOLD, SELL, STRONG SELL) is then derived from the predicted direction, predicted return magnitude, calibrated confidence and outperformance versus baseline. Borderline trades are downgraded to HOLD when confidence is low or when the model has been losing to the baseline.

4.3.6 Backtesting and Portfolio Optimisation

Two further modules are exposed through the same Flask app. The backtester (`backtester.py`) runs an expanding window walk forward backtest, retraining the ensemble every thirty trading days and simulating a long only strategy with a buy threshold of +0.3 percent and a sell threshold of -0.2 percent. Results include Sharpe ratio, alpha, maximum drawdown, win rate, profit factor and an equity curve. The portfolio optimiser (`portfolio_optimizer.py`) implements Markowitz mean variance optimisation through SciPy's SLSQP solver, returning the maximum Sharpe portfolio, the minimum variance portfolio, a balanced allocation and the efficient frontier for any user supplied basket of symbols.

4.4 Frontend Design

4.4.1 Application Structure

The React frontend is a single page application with hash based routing. The App component manages global state, including the authenticated user, the current route, the selected stock symbol, live price data, historical data, predictions and portfolio state. Child components receive data and callbacks as props.

The application renders different page level components based on the current route. The principal pages are Overview (the landing page), Dashboard (the main trading workspace), Live Markets (candlestick charts), Analytics Hub (aggregate charts), Research Hub, Settings, Profile and an Auth page for login and registration.

4.4.2 Dashboard Layout

The Dashboard page is the core of the application. It is laid out as a series of two column rows using CSS Grid. The left panel contains the PopularStocks grid, a search bar and a watchlist. The right panel contains the historical price chart with the prediction overlay. Below this, the left panel shows the StockInfo card, PredictionCard and TradingPanel, while the right panel displays the PerformanceChart

and Portfolio summary. A third row holds the TradeHistory and NewsSentiment components.

This layout was designed to mirror the information density of professional trading terminals while remaining readable and navigable on smaller screens. At viewport widths below 1100 pixels, the two column layout collapses to a single column.

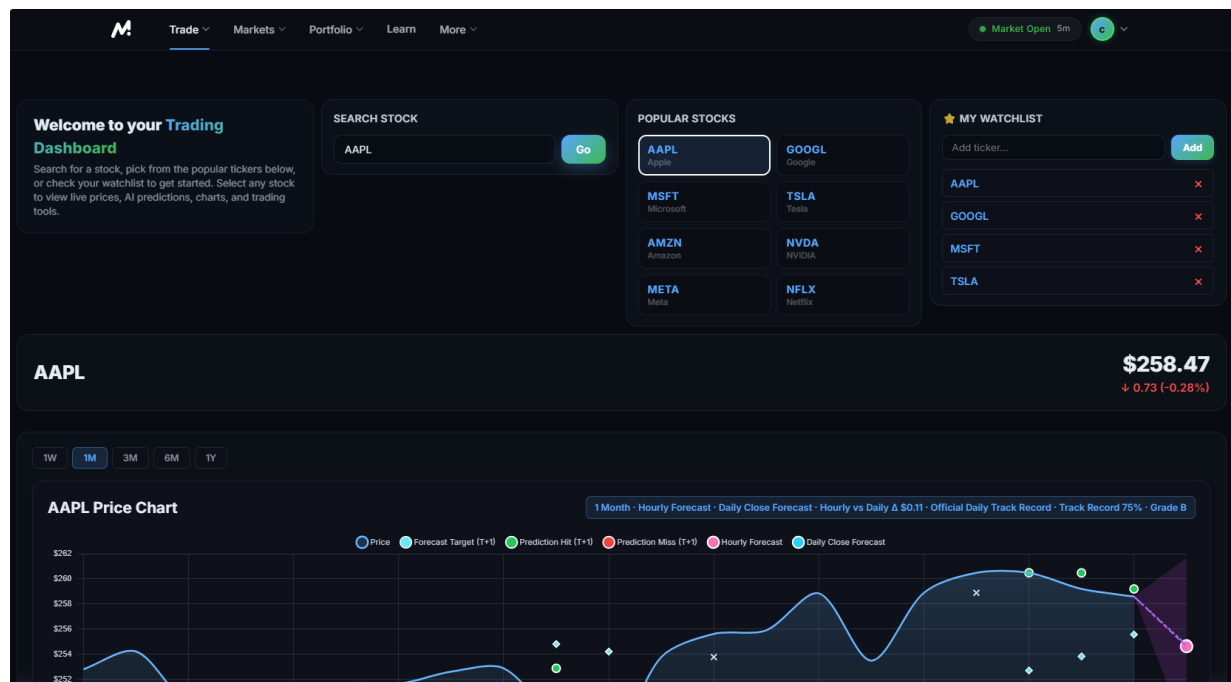


Figure 4.4: MarketMind Dashboard with Live Stock Data.

Figure 4.4 shows the dashboard loaded with live stock data, including the historical price chart, prediction card and trading panel.

4.4.3 State Management

Global state is managed using React's `useState` and `useEffect` hooks at the App component level. There is no external state management library such as Redux. This decision was made because the state is not deeply nested, is not shared across many unrelated components and does not require complex update logic. Prop drilling is used to pass state and callbacks to child components.

Authentication state is persisted in `localStorage`. On initial load, the app checks for a stored JWT, validates it against the `/api/auth/me` endpoint and restores

the session if the token is still valid. Settings such as theme preference, auto refresh and compact mode are also persisted in `localStorage`.

4.4.4 Data Visualisation

`Chart.js` is used throughout the application for data visualisation. The `HistoricalChart` component renders a line chart of closing prices with a gradient fill beneath the line. When a prediction is available, a horizontal dashed line is drawn at the predicted price level, giving the user a visual reference for the model's forecast relative to recent price history.

The `CandleChart` component renders candlestick data for the Live Markets page. Each candle shows the open, high, low and close for a given time interval, with green candles indicating bullish periods and red candles indicating bearish ones. The Live Markets page also embeds a `TradingView` widget for users who want full professional grade charting alongside the rest of the dashboard.

The `AnalyticsHub` uses `Chart.js` line charts for liquidity scores, bar charts for sector fund flows and a doughnut chart for factor rotation data.

4.5 Database Design

The PostgreSQL database contains seven tables that fall into three groups: authentication, trading and prediction logging.

The `users` table stores user account data with columns for `id` (auto generated primary key), `email` (unique, not null), `password_hash` (not null), `display_name` (not null), `role` (an enum of `USER` or `ADMIN`), `created_at` and `updated_at`.

The trading group consists of four tables. The `trades` table records every executed buy or sell with columns for `id`, `user_id`, `symbol`, `type`, `price`, `quantity`, `total_value`, `pnl` and `timestamp`. The `holdings` table tracks open positions with columns for `user_id`, `symbol`, `quantity` and `average cost basis`. The `user_cash_balance` table stores each user's virtual cash. The `portfolio_snapshots` table stores periodic snapshots of total portfolio value and is used to render the equity curve in the frontend.

The `prediction_logs` table stores prediction records with columns for `id`, `symbol`, `model`, `predicted_price`, `current_price`, `confidence`, `direction`, `horizon_hours`, `actual_price` (nullable until evaluated), `absolute_error` (nullable), `percent_error` (nullable), `direction_correct` (nullable boolean), `created_at` and `evaluated_at` (nullable).

Two additional tables support the educational module: `learning_progress` tracks lesson completion per user, and `user_xp` stores experience points and streak data for the gamified learning experience.

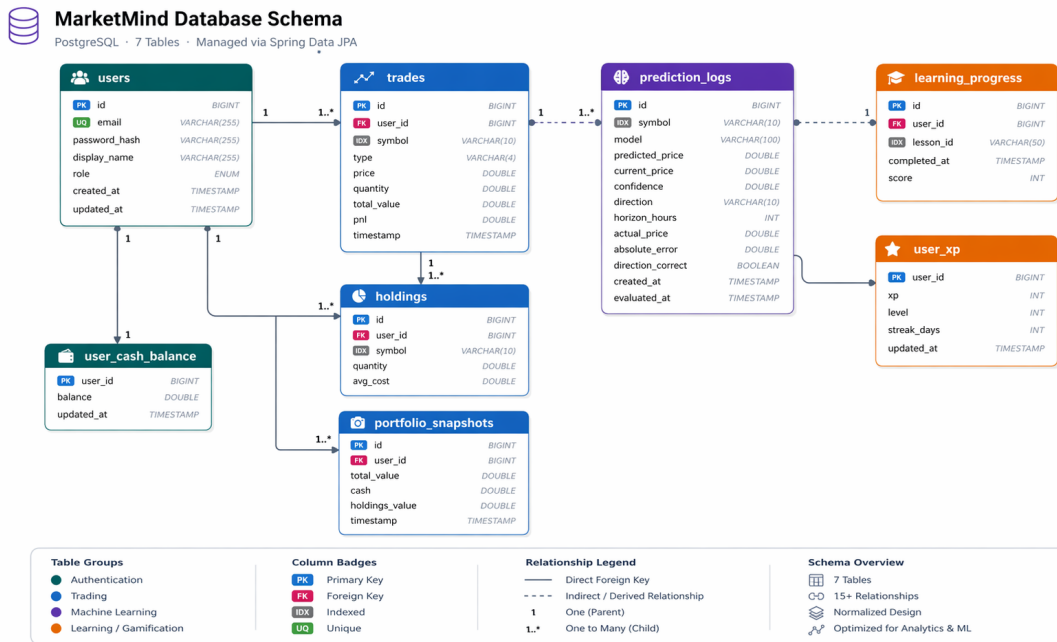


Figure 4.5: Database Schema.

Spring Data JPA manages the object relational mapping. The entity classes are annotated as JPA entities and their corresponding repositories extend Spring's `JpaRepository` interface to provide CRUD operations and custom query methods without writing SQL. Figure 4.5 illustrates the database schema.

4.6 Security Design

Security was treated as a first class concern throughout the design. Passwords are never stored in plaintext. BCrypt hashing with a work factor of 10 (the Spring Security default) is used for all password storage. JWT tokens are signed with a configurable secret key and include an expiry timestamp. The `JwtAuthenticationFilter` rejects expired or malformed tokens with a 401 response.

The `SecurityConfig` class defines the security filter chain. Public endpoints (registration, login and static resources) are permitted without authentication. All other endpoints require a valid JWT. CORS is configured to allow requests from the frontend origin during development.

Role based access control is supported through the `UserRole` enum, which currently defines `USER` and `ADMIN` roles. This provides a foundation for restricting

access to administrative features such as viewing all users, resetting accounts or accessing aggregate prediction metrics.

4.7 Deployment Architecture

The project includes Dockerfiles for the backend and ML service, and a Docker Compose configuration that orchestrates all three services plus a PostgreSQL database container. The `dev.sh` script provides a non Docker alternative for local development, starting all services in parallel with a single command and shutting them down cleanly on exit.

The platform is deployed to Microsoft Azure and is accessible live at `https://marketmind.cfd`. A GitHub Actions CI/CD pipeline automates testing on every push to the main branch, ensuring that the codebase remains in a deployable state throughout the project.

Environment variables are managed through a `.env` file at the repository root. Required variables include the Finnhub API key and NewsAPI key. Optional variables include the JWT signing secret, database connection details and frontend URL overrides.

Chapter 5

System Evaluation

This chapter evaluates the completed MarketMind platform against the objectives set out in the Introduction. It presents quantitative results from the prediction model, discusses the strengths and weaknesses of the system and reflects on the ethical and practical limitations of the approach.

5.1 Evaluation Against Objectives

Each of the ten objectives defined in Chapter 1 is addressed below.

Objective 1: Java Spring Boot REST API. The backend was fully implemented using Spring Boot 3.2 with Java 17. It exposes RESTful endpoints for stock data retrieval, trading operations, portfolio management, authentication, prediction logging, administrative oversight and the educational module. The API follows consistent naming conventions, returns JSON responses and handles errors gracefully with appropriate HTTP status codes. This objective was met in full.

Objective 2: External API integration. Three external APIs were integrated successfully. Finnhub provides real time stock quotes. Yahoo Finance supplies historical daily price data. NewsAPI delivers financial news headlines. Each integration is encapsulated in a dedicated client or service class within the backend, making it straightforward to swap out a provider if needed. This objective was met in full.

Objective 3: PostgreSQL database. A PostgreSQL database was designed and implemented to persist user accounts, trade records, holdings, cash balances, portfolio snapshots, prediction logs and educational progress. Spring Data JPA manages the object relational mapping across all seven tables, and the schema supports the full authentication, trading and prediction evaluation workflows. This objective was met in full.

Objective 4: Machine learning prediction model. A Python based pre-

diction model was implemented and deployed as a Flask microservice. The model combines a three way ensemble of Random Forest, Gradient Boosting and XGBoost regressors with FinBERT powered sentiment fusion, a stability guardrail and calibrated confidence scoring. It predicts five day forward returns trained on 29 engineered features spanning technical indicators, sentiment and macro context. This objective was met in full.

Objective 5: ML-to-backend integration. The Java backend communicates with the Python ML service over HTTP. The MLController requests predictions from the Flask service, logs them in the database and returns the results to the frontend. The PredictionScheduler automates this process on a configurable interval. This objective was met in full.

Objective 6: Demo trading module. The TradingService supports simulated buy and sell operations against a virtual cash balance. Positions are tracked, and portfolio value is calculated dynamically using current market prices. Users can view their open positions, total portfolio value and trade history. This objective was met in full.

Objective 7: Responsive React dashboard. The frontend delivers a comprehensive dashboard with live prices, historical charts, prediction cards, news sentiment, candlestick charts, portfolio summaries and trade history. The interface is responsive and adapts to desktop, tablet and mobile viewports. Both dark and light themes are supported. This objective was met in full.

Objective 8: Secure authentication. User authentication is implemented using BCrypt password hashing and JWT session tokens. Registration, login, token validation and profile updates all work correctly. Role based access control is supported through the UserRole enum. This objective was met in full.

Objective 9: Prediction evaluation. The PredictionLogService evaluates predictions whose time horizon has elapsed by comparing the predicted price to the actual price at evaluation time. Metrics including mean absolute error, mean percentage error, directional accuracy and mean absolute percentage error are computed and exposed through the API, and the `/results` page compares the ensemble against Always-Up, Always-Down and one-step persistence baselines so that headline figures are always placed in context. This objective was met in full.

Objective 10: Documentation. This dissertation documents the design, architecture, methodology and evaluation of the platform. The GitHub repository includes a comprehensive README with installation instructions, API documentation and a project screencast. This objective is met by the submission of this document.

5.2 Prediction Model Results

The prediction model was evaluated using a 150 trading day backtest spanning approximately seven months from September 2025 to April 2026. Predictions were generated for eight actively traded US large caps: AAPL, MSFT, TSLA, AMZN, NVDA, META, NFLX and GOOGL. For every prediction the realised closing price at the forecast horizon was recorded and compared both to the ensemble output and to a one step persistence baseline that forecasts the next close equal to the current close, following the naive benchmark methodology recommended by Hyndman and Athanasopoulos [3]. A snapshot of the full results set is persisted at `ml-service/live_model_results.json` and visualised on the `/results` page of the running application, so that every figure in this chapter can be verified independently.

Table 5.1 presents the headline metrics per symbol and in aggregate.

Symbol	Dir. Acc. (%)	Ensemble MAPE (%)	Baseline MAPE (%)	Δ vs Baseline
AAPL	53.33	3.42	2.61	−31.3%
MSFT	48.00	3.37	2.98	−13.0%
TSLA	52.00	6.24	4.11	−51.6%
AMZN	50.67	4.69	3.78	−23.9%
NVDA	55.33	3.86	3.53	−9.3%
META	45.64	5.80	4.18	−38.9%
NFLX	46.00	6.01	4.07	−47.8%
GOOGL	44.67	4.82	3.48	−38.6%
Average	49.54	4.78	3.59	−31.6%

Table 5.1: Prediction model performance over the 150 day backtest (Sep 2025 – Apr 2026). The Δ vs Baseline column reports the ensemble’s mean absolute percentage error relative to the persistence baseline; negative values indicate the ensemble is worse than the baseline on magnitude.

The picture these numbers paint is deliberately an honest one. Average directional accuracy is 49.54 percent, being essentially a coin flip, and the ensemble’s average MAPE of 4.78 percent is approximately 31.6 percent worse than the persistence baseline of 3.59 percent. Directional accuracy ranges from 44.67 percent on GOOGL up to 55.33 percent on NVDA, and only four of the eight symbols finish above the 50 percent threshold. Magnitude error follows the expected shape: it is lowest on the more stable large caps (MSFT at 3.37 percent, AAPL at 3.42 percent) and highest on the more volatile names (TSLA at 6.24 percent, NFLX at 6.01 percent).

These results need to be read in context. The model forecasts short horizon returns on daily tabular features, and over a seven month window spanning several

distinct market regimes a tree based ensemble with sentiment fusion is being asked to do something the academic literature repeatedly warns is extremely difficult [27]. The outcome is consistent with that literature. It is also the reason the confidence score surfaced in the API is bounded between 35 and 94 percent and is calibrated against validation error rather than displayed as a raw probability: the model is prevented from claiming certainty that the evaluation data does not support. The symbols whose forecasts deserve the least trust (TSLA, AAPL, AMZN, META, NFLX, GOOGL in the snapshot above) are correspondingly the ones whose confidence scores remain pinned at or near the 35 percent floor, and the user interface uses that score to down-weight the signal shown in the prediction card.

Rather than hide these numbers, the platform exposes them. The `/results` page renders the same backtest as a predicted-versus-actual line chart, a hit-rate comparison against Always-Up, Always-Down and persistence baselines and a per-regime breakdown. A reader of this dissertation can verify every figure in Table 5.1 against the live page or against the JSON snapshot checked in at `ml-service/live_model_results.json`. This tamper-evident accountability loop is, in the author's view, a more valuable contribution than an inflated headline figure would have been.

5.3 Strengths

Several strengths of the system are worth highlighting.

The microservices architecture proved to be a significant advantage during development. Being able to restart the ML service independently of the backend, or to test frontend changes without touching the backend, saved considerable time and reduced the risk of introducing regressions.

The authentication system is robust and follows industry best practices. BCrypt hashing, JWT tokens with expiry and Spring Security's filter chain provide a solid security foundation that could support a production deployment with only minor hardening (such as switching to HTTPS and rotating the JWT secret).

The frontend delivers a polished, professional user experience. The dark theme, smooth animations, responsive layout and comprehensive feature set compare favourably with commercial trading dashboards. The dual theme support and settings page demonstrate attention to user preferences and accessibility.

The prediction logging and evaluation system provides genuine insight into model performance. Rather than simply displaying predictions and moving on, the system records every prediction, evaluates it after the horizon elapses and computes aggregate metrics. This accountability loop is rare in demo trading platforms and adds significant educational value.

The CI/CD pipeline with GitHub Actions ensures code quality is maintained throughout the development process. Every push triggers automated tests, catching issues before they reach the main branch.

5.4 Limitations

The system has several limitations that should be acknowledged.

The ensemble does not beat a one-step persistence baseline on mean absolute percentage error. Over the 150 day evaluation the baseline is on average 31.6 percent more accurate on magnitude, and the ensemble's directional accuracy averages 49.54 percent. This is a genuine limitation of the modelling approach, not a reporting artefact, and it is the central reason the platform emphasises confidence bounding, baseline comparison and transparent accountability rather than headline precision. It also aligns with the well documented difficulty of beating naive benchmarks on short horizon equity forecasting [27].

The prediction model, although a substantial step beyond a single regressor, is still bounded by what tree based ensembles can learn from daily tabular features. It cannot model the long range temporal dependencies that a sequence model such as an LSTM or transformer would capture, and it does not exploit intraday or order book level data. Adding a deep learning branch to the ensemble would likely improve performance, particularly on volatile names where regime shifts are harder to anticipate from daily features alone, but doing so would require GPU resources and training time well beyond the scope of this project.

The Finnhub free tier imposes a rate limit of 60 API calls per minute. During periods of heavy use or when multiple users are requesting prices simultaneously, this limit could be reached. Caching frequently requested quotes with a short time to live mitigates this in practice but does not eliminate it.

The platform does not currently support WebSocket connections for real time price streaming. Prices are fetched on demand or at configurable polling intervals, which introduces a small delay between market movements and what is shown in the dashboard. A WebSocket integration with Finnhub's streaming API on a paid tier would close this gap.

The Reddit sentiment pipeline relies on Reddit's unauthenticated public JSON API, which is rate limited and occasionally returns 429 responses. The service handles this by backing off and retrying, but the limit means that very heavy use could temporarily degrade the social sentiment signal until the rate window clears.

The walk forward backtester uses a long only strategy with fixed buy and sell thresholds. A more sophisticated implementation would include short selling, position sizing based on confidence, transaction costs and slippage. These were left out deliberately to keep the backtest interpretable, but they limit how far

the reported Sharpe and alpha figures can be read as predictors of real world performance.

5.5 Usability

The platform was informally tested with three peers who had varying levels of familiarity with stock trading. All three were able to register, log in, search for a stock, view its price and prediction and execute a simulated trade without guidance. Two of the three commented positively on the visual design and the clarity of the prediction card. One user noted that the distinction between the historical chart and the prediction overlay could be made clearer with a legend or annotation.

No formal usability study was conducted due to time constraints, but the informal feedback suggests that the interface is intuitive and accessible to users without prior trading experience.

5.6 Ethical Considerations

The ethical dimensions of the platform deserve careful consideration. Stock price prediction is inherently uncertain, and any prediction model carries the risk that users will interpret its output as reliable trading advice. MarketMind addresses this risk in several ways.

The platform operates exclusively with virtual funds. There is no mechanism to connect it to a real brokerage or to execute real trades. This is made explicit through disclaimers on the landing page, the footer and within the trading panel itself.

The prediction model's limitations are described transparently. The model architecture, the FinBERT sentiment fusion, the stability guardrail and the calibrated confidence are all surfaced through the API and shown alongside every prediction in the user interface. The confidence score is bounded between 35 and 94 percent and is calibrated against validation error and historical hit rate, so the model never claims certainty it cannot defend. Lower confidence scores for volatile stocks and for predictions where ensemble members disagree provide an honest signal that the forecast should be treated with caution.

The platform does not collect sensitive personal data. Only email addresses and display names are stored. Passwords are hashed and never logged. JWT tokens expire after a configurable period, limiting the window of risk if a token is compromised.

5.7 Performance

Response times were measured informally during development. The backend's stock price endpoint typically responds within 200 to 400 milliseconds, with the majority of that time attributable to the Finnhub API call. The prediction endpoint takes 1 to 3 seconds, as it requires the ML service to fetch historical data from Yahoo Finance before computing the prediction. The authentication endpoints (register and login) respond within 100 milliseconds.

The frontend renders the dashboard within 500 milliseconds of receiving data from the backend. Chart.js animations add a brief visual delay but do not affect the perceived responsiveness of the interface.

These response times are acceptable for a demo trading platform but would need optimisation for a production system serving many concurrent users. Caching, connection pooling and asynchronous data fetching would be the primary areas for improvement.

Chapter 6

Conclusion

6.1 Summary

This project set out to design, develop and deploy a machine learning-based predictive trading and simulation platform that would bring institutional-grade market analysis tools to a broader audience. The result, MarketMind, is a full-stack web application that integrates real-time stock data, historical price analysis, machine learning price predictions, news sentiment analysis and paper trading into a responsive, professional dashboard.

The platform was built using a microservices architecture comprising three independently deployable services. A Java Spring Boot backend handles authentication, data aggregation, trade execution and prediction logging. A Python Flask microservice runs the prediction model. A React frontend delivers the user-facing dashboard with live charts, prediction cards, a trading panel and portfolio tracking.

All ten objectives defined at the outset of the project were met. The prediction model demonstrates the full pipeline from five years of historical data through feature engineering, ensemble training, sentiment fusion, guardrails, calibrated confidence and automated evaluation. The authentication system follows industry best practices with BCrypt hashing and JWT session management. The frontend provides a polished, responsive experience with dark and light themes, comprehensive data visualisation, an interactive backtester, a Markowitz portfolio optimiser and a thoughtful settings page.

6.2 Key Findings

All ten objectives set out in Chapter 1 were delivered. Objectives 1, 5, 6 and 8 produced a cohesive, secure backend that handles authentication, trading, portfolio tracking and ML integration over a clean REST API. Objectives 2, 3 and 7

produced the data and presentation layers that feed the frontend with live quotes, historical prices, news, sentiment and a seven table PostgreSQL store. Objectives 4 and 9 produced the prediction model itself, evaluated below.

The prediction model was evaluated against a one-step persistence baseline over a 150 trading day backtest from September 2025 to April 2026 across eight US large caps. Average directional accuracy was 49.54 percent and the ensemble's average MAPE of 4.78 percent was 31.6 percent worse than the persistence baseline of 3.59 percent. Only four of the eight symbols finished above the 50 percent line on direction, and magnitude error was lowest on the stable large caps and highest on the volatile names. Rather than hide these numbers, the platform exposes them at the `/results` page and persists a JSON snapshot in the repository so every figure in Table 5.1 can be checked independently. That accountability loop is the more useful contribution here, not a flattering headline number.

The microservices architecture was a good fit for this project. Each service could be developed, tested and deployed on its own, bugs and config changes were contained and the data, intelligence and presentation layers stayed cleanly separated.

The technology stack of Spring Boot, Flask, React and PostgreSQL delivered a robust, maintainable system that could realistically be extended into a production platform. All of these are widely used in the Irish and European software industry, and working with them across the full stack of the project was a serious practical skills uplift.

6.3 Reflection

Looking back, several things went well. Investing early in the authentication system and the REST API structure meant that new features were built on a stable foundation rather than bolted onto a fragile prototype. The `dev.sh` script that starts all three services with a single command saved a lot of time during development and is a small but useful example of good developer experience.

Most of the learning happened in the prediction model. The first version was a weighted moving average with trend and momentum components. It worked end to end as a pipeline but produced predictions that were far too smooth to be useful. Rebuilding it around a Random Forest, Gradient Boosting and XGBoost ensemble trained on five day forward returns, with FinBERT sentiment fusion and a stability guardrail, made every part of the pipeline harder to reason about but pushed the work into the parts of applied ML that papers tend to glide over: leakage prevention, feature scaling order, validation-driven ensemble weighting and confidence calibration. The most important lesson came out of the 150 day backtest: beating a one-step persistence baseline on short horizon equity data is genuinely difficult,

and the right response is to publish the comparison rather than pick a friendlier metric.

If the project were started again, two things would be done differently. First, the prediction logging infrastructure would go in from day one rather than being added later. A longer history of evaluated predictions would make the confidence calibration more honest from the start. Second, the ML service would have had a proper unit test suite from the beginning. The current tests cover the main code paths, but a more rigorous harness around the feature engineering layer, where small bugs are hard to spot by eye, would have caught a few subtle issues earlier.

6.4 Future Work

Several avenues for future work have been identified through the development and evaluation process.

A deep learning branch could be added to the existing ensemble. An LSTM or transformer based model trained on sequences of daily features would capture longer range temporal dependencies that the current tree based ensemble cannot. The Flask serving infrastructure already supports adding new model paths, and the inverse MAE weighting scheme would naturally fold the new model into the existing ensemble.

The evaluation window could be extended further. The current 150 day back-test covers seven months and several distinct regimes, but a multi-year window across a broader basket would give a clearer picture of how the ensemble holds up across full market cycles. The prediction logging infrastructure is already in place to support this without code changes.

WebSocket streaming for real time price updates would replace the current polling model on the Live Markets page, removing the small delay between market movements and the dashboard. Finnhub offers a WebSocket API on its paid tier that could be integrated without major architectural changes.

The walk forward backtester could be extended to support short selling, position sizing scaled by predicted confidence, transaction costs and slippage. These would make the reported performance figures more realistic and would let the platform compare different strategy configurations rather than only the current long only one.

Multi factor authentication and OAuth integration, allowing users to log in with Google or GitHub accounts, would further strengthen the security posture of the platform and reduce the friction of registration.

A formal usability study with a larger participant pool would provide more rigorous feedback on the dashboard than the informal testing carried out during development. Particular attention could be paid to whether the calibrated con-

fidence scores and the regime tags are interpreted correctly by users without a finance background.

6.5 Applied Learning Outcomes

The project maps to the module learning outcomes as follows.

Research methodologies for software development. The prediction model went through a deliberate prototype-then-rebuild cycle (a weighted SMA proof of concept, then a tree based ensemble), the technology selections were grounded in a literature review, and strategy performance was measured with a walk forward backtest against a published baseline methodology.

Awareness of the state of the art. Chapter 3 surveys the relevant tools and reasons explicitly about why FinBERT, XGBoost and ensemble learning were chosen over the simpler options that were tried first.

Integration of disparate technologies. The platform combines a Java Spring Boot backend, a Python Flask ML service, a React frontend, PostgreSQL, scikit-learn, XGBoost, HuggingFace Transformers, Docker and GitHub Actions into one end to end deployment.

Research and critical thinking on a challenging problem. Chapter 4 documents the design decisions on return targets, leakage prevention, ensemble weighting, stability guardrails and confidence calibration, each tied back to the underlying literature.

Evaluation, selection and application of research tools. The methodology chapter records the comparison between modelling approaches, and the continuous learning pipeline uses GridSearchCV with TimeSeriesSplit to keep hyperparameter tuning leakage-free.

Design and implementation of a computing solution requiring preliminary research. This is the core deliverable.

Critical evaluation, reflection and future potential. Chapter 5 reports the evaluation honestly, including the ensemble's failure to beat the persistence baseline, and Sections 6.3 and 6.4 reflect on what was learned and what should come next.

6.6 Closing Remarks

MarketMind shows that one student can build, deploy and honestly evaluate a microservices trading platform inside the scope of a final year project. The system pulls together Java, Python and JavaScript stacks, four external data sources, a FinBERT sentiment pipeline, a tree based ensemble and a walk forward backtester, and it does so without hiding the parts that did not work as well as hoped. The

habits picked up along the way, particularly comparing the model against a published baseline rather than against itself, carry forward directly into professional software work.

The source code, documentation and a screencast demonstration are available at <https://github.com/michaelferry25/MachineLearningBasedPredictiveTradingPlatform>.

Bibliography

- [1] Finnhub. Finnhub stock api. <https://finnhub.io/docs/api>, 2024. Accessed: 2025-10-15.
- [2] Yahoo. Yahoo finance api. <https://finance.yahoo.com>, 2024. Accessed: 2025-10-20.
- [3] Rob J. Hyndman and George Athanasopoulos. *Forecasting: Principles and Practice*. OTexts, Melbourne, Australia, 3rd edition, 2021. <https://otexts.com/fpp3/>.
- [4] JetBrains. The state of developer ecosystem 2024. <https://www.jetbrains.com/lp/devecosystem-2024/>, 2024. Accessed: 2025-10-25.
- [5] Oracle. Jdk 17 release notes. <https://www.oracle.com/java/technologies/javase/17-relnote-issues.html>, 2021. Accessed: 2025-09-15.
- [6] VMware. Spring boot reference documentation. <https://spring.io/projects/spring-boot>, 2024. Accessed: 2025-09-25.
- [7] VMware. Spring security reference. <https://spring.io/projects/spring-security>, 2024. Accessed: 2025-10-10.
- [8] Niels Provos and David Mazières. A future-adaptable password scheme. *Proceedings of the USENIX Annual Technical Conference*, pages 81–91, 1999.
- [9] The PostgreSQL Global Development Group. Postgresql documentation. <https://www.postgresql.org/docs/>, 2024. Accessed: 2025-10-01.
- [10] Gradle Inc. Gradle build tool. <https://gradle.org>, 2024. Accessed: 2025-09-28.
- [11] Meta. React documentation. <https://react.dev>, 2024. Accessed: 2025-09-20.

- [12] Evan You. Vite: Next generation frontend tooling. <https://vitejs.dev>, 2024. Accessed: 2025-09-22.
- [13] Chart.js Contributors. Chart.js documentation. <https://www.chartjs.org>, 2024. Accessed: 2025-10-05.
- [14] Python Software Foundation. Python 3.11 documentation. <https://docs.python.org/3.11/>, 2024. Accessed: 2025-09-18.
- [15] Armin Ronacher. Flask documentation. <https://flask.palletsprojects.com>, 2024. Accessed: 2025-10-08.
- [16] Thomas G. Dietterich. Ensemble methods in machine learning. In *International workshop on multiple classifier systems*, pages 1–15. Springer, 2000.
- [17] Leo Breiman. Random forests. *Machine learning*, 45(1):5–32, 2001.
- [18] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, et al. Scikit-learn: Machine learning in python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.
- [19] Jerome H. Friedman. Greedy function approximation: a gradient boosting machine. *Annals of statistics*, pages 1189–1232, 2001.
- [20] Tianqi Chen and Carlos Guestrin. Xgboost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 785–794, 2016.
- [21] Gerald Appel. *Technical analysis: power tools for active investors*. FT Press, Upper Saddle River, NJ, 2005.
- [22] J. Welles Wilder. *New concepts in technical trading systems*. Trend Research, Greensboro, NC, 1978.
- [23] Dogu Araci. Finbert: Financial sentiment analysis with pre-trained language models. *arXiv preprint arXiv:1908.10063*, 2019.
- [24] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805*, 2018.
- [25] Steven Loria. Textblob: Simplified text processing. <https://textblob.readthedocs.io>, 2024. Accessed: 2025-11-15.

- [26] Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Rémi Louf, Morgan Funtowicz, et al. Transformers: State-of-the-art natural language processing. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 38–45, 2020.
- [27] David H. Bailey, Jonathan M. Borwein, Marcos López de Prado, and Qiji Jim Zhu. Pseudo-mathematics and financial charlatanism: The effects of backtest overfitting on out-of-sample performance. *Notices of the American Mathematical Society*, 61(5):458–471, 2014.
- [28] Harry Markowitz. Portfolio selection. *The Journal of Finance*, 7(1):77–91, 1952.
- [29] Pauli Virtanen, Ralf Gommers, Travis E. Oliphant, Matt Haberland, Tyler Reddy, David Cournapeau, Evgeni Burovski, Pearu Peterson, Warren Weckesser, Jonathan Bright, et al. Scipy 1.0: fundamental algorithms for scientific computing in python. *Nature methods*, 17(3):261–272, 2020.
- [30] William F. Sharpe. Mutual fund performance. *The Journal of Business*, 39(1):119–138, 1966.
- [31] NewsAPI. Newsapi documentation. <https://newsapi.org/docs>, 2024. Accessed: 2025-11-01.
- [32] Docker Inc. Docker documentation. <https://docs.docker.com>, 2024. Accessed: 2025-11-05.
- [33] GitHub. Github actions documentation. <https://docs.github.com/en/actions>, 2024. Accessed: 2025-11-10.

Appendix A

Appendix

A.1 GitHub Repository

The complete source code for the MarketMind platform is available at the following URL:

`https://github.com/michaelferry25/MachineLearningBasedPredictiveTradingPlatform`

A.2 Installation Instructions

The following instructions describe how to compile, deploy and run the project on a local machine.

A.2.1 Prerequisites

- Node.js 18 or higher
- Java Development Kit 17
- Python 3.11 or higher
- npm package manager
- A Finnhub API key (free tier available at `https://finnhub.io`)
- A NewsAPI key (free tier available at `https://newsapi.org`)

A.2.2 Clone the Repository

```
git clone https://github.com/michaelferry25/  
    MachineLearningBasedPredictiveTradingPlatform.git  
cd MachineLearningBasedPredictiveTradingPlatform
```

A.2.3 Configure Environment Variables

Copy the example environment file and add your API keys:

```
cp .env.example .env
```

Edit the .env file and set the following variables:

```
FINNHUB_API_KEY=your_finnhub_key_here  
NEWS_API_KEY=your_newsapi_key_here
```

A.2.4 Quick Start (All Services)

A convenience script starts all three services simultaneously:

```
./dev.sh
```

This launches the backend on port 8080, the ML service on port 5001 and the frontend on port 5173.

A.2.5 Manual Setup

To start each service individually:

Backend:

```
cd backend  
./gradlew build  
./gradlew bootRun
```

ML Service:

```
cd ml-service  
python -m venv venv  
source venv/bin/activate  
pip install -r requirements.txt  
python app.py
```

Frontend:

```
cd frontend  
npm install  
npm run dev
```


A.2.6 Using the Application

Open a browser and navigate to `http://localhost:5173`. Create an account or log in, select a stock from the popular stocks grid or search by ticker symbol, view real-time prices and predictions and use the trading panel to execute simulated buy and sell orders.

A.3 API Endpoints

Method	Endpoint	Description
GET	/api/price/{symbol}	Fetch current stock price
GET	/api/historical/{symbol}	Retrieve historical prices
GET	/api/news/{symbol}	Get news with sentiment
GET	/api/candles/{symbol}	Get candlestick data
GET	/api/ml/predict/{symbol}	Get ML price prediction
GET	/api/ml/metrics	Get prediction accuracy metrics
POST	/api/trading/buy	Execute simulated buy order
POST	/api/trading/sell	Execute simulated sell order
GET	/api/trading/portfolio	Fetch portfolio summary
GET	/api/trading/history	Retrieve trade history
POST	/api/auth/register	Register a new account
POST	/api/auth/login	Authenticate and receive JWT
GET	/api/auth/me	Get current user profile
PUT	/api/auth/profile	Update display name

Table A.1: MarketMind API Endpoint Summary

A.4 Screenshot

A brief screencast demonstrating the application's features is available in the GitHub repository.